

# Shiv Nadar University Chennai

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27 |

## Experiment 1

### Implementation of a Single Layer Perceptron for Binary Classification

#### 1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

#### 2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

#### Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$  : Input vector
- $\mathbf{w}$  : Weight vector
- $b$  : Bias
- $z$  : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where  $\eta$  denotes the learning rate.

### 3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

#### Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

| Activation | Output Range        | Typical Applications                    |
|------------|---------------------|-----------------------------------------|
| Step       | $\{0, 1\}$          | Classical Perceptron                    |
| Sigmoid    | $(0, 1)$            | Binary Classification Output Layer      |
| Tanh       | $(-1, 1)$           | Hidden Layers (Traditional Networks)    |
| ReLU       | $[0, \infty)$       | Hidden Layers in CNNs and MLPs          |
| Leaky ReLU | $(-\infty, \infty)$ | Avoiding Dying ReLU                     |
| Softmax    | $(0, 1)$            | Multi-class Classification Output Layer |

**Note:** This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

### 4. Dataset Description

**Dataset:** Banknote Authentication Dataset

**Source:** UCI Machine Learning Repository

**Download Link:**

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

|                |                       |
|----------------|-----------------------|
| Instances      | 1372                  |
| Features       | 4 Numerical Features  |
| Classes        | 2                     |
| Missing Values | None                  |
| Task           | Binary Classification |

### Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

## 5. Experimental Procedure

### Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

#### Expected Output

Dataset summary and statistical information.

### Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

### Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

#### **Task 4: Perceptron Implementation**

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

#### **Task 5: Model Training**

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

#### **Task 6: Model Evaluation**

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

## **6. Mandatory Plots**

| <b>Plot</b>                                                                                         | <b>Purpose</b>        | <b>Expected Inference</b>       |
|-----------------------------------------------------------------------------------------------------|-----------------------|---------------------------------|
| Histogram_Variance.eps<br>Histogram_Skewness.eps<br>Histogram_Curtosis.eps<br>Histogram_Entropy.eps | Distribution Analysis | Identify skewness and outliers. |

|                                                                                             |                        |                                                                 |
|---------------------------------------------------------------------------------------------|------------------------|-----------------------------------------------------------------|
| Boxplot_Variance.eps<br>Boxplot_Skewness.eps<br>Boxplot_Curtosis.eps<br>Boxplot_Entropy.eps | Distribution Analysis  | Compare quartiles, medians, and outliers.                       |
| Correlation_Heatmap.eps                                                                     | Feature Relationship   | Observe highly correlated features.                             |
| Scatter_Plot.eps                                                                            | Class Distribution     | Determine whether classes are approximately linearly separable. |
| Training_Error_vs_Epoch.eps                                                                 | Learning Behaviour     | Verify convergence of the perceptron.                           |
| Weight_Evolution.eps                                                                        | Parameter Analysis     | Observe stabilization of learned weights.                       |
| Bias_Evolution.eps                                                                          | Parameter Analysis     | Understand the role of the bias term.                           |
| Confusion_Matrix.eps                                                                        | Performance Evaluation | Interpret TP, FP, TN and FN.                                    |
| Learning_Rate_Comparison.eps                                                                | Hyperparameter Study   | Compare convergence behaviour for different learning rates.     |

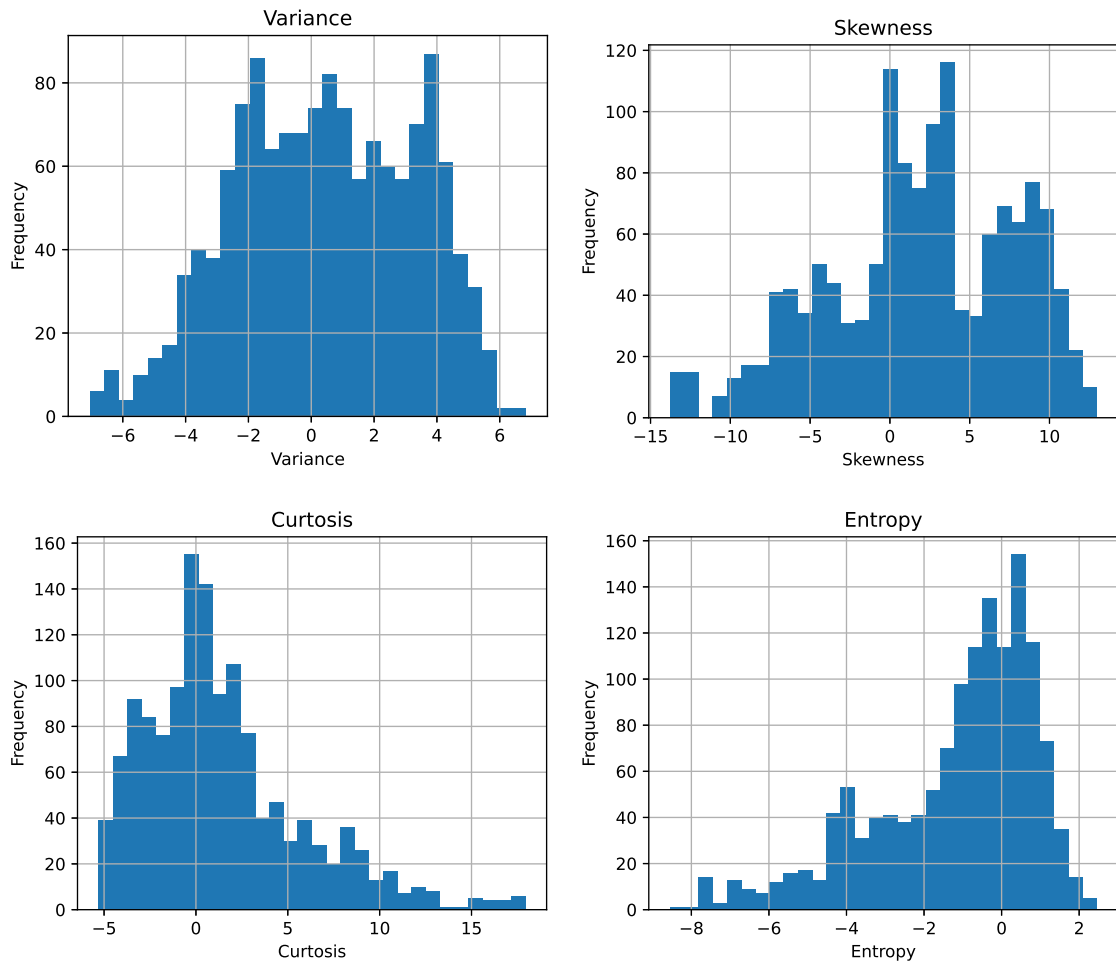


Figure 1: Feature Histograms: Illustrates the distribution of each feature. Authentic banknotes show distinct spread in variance and skewness compared to forged ones, helping identify distinguishing factors and skewness in the data.

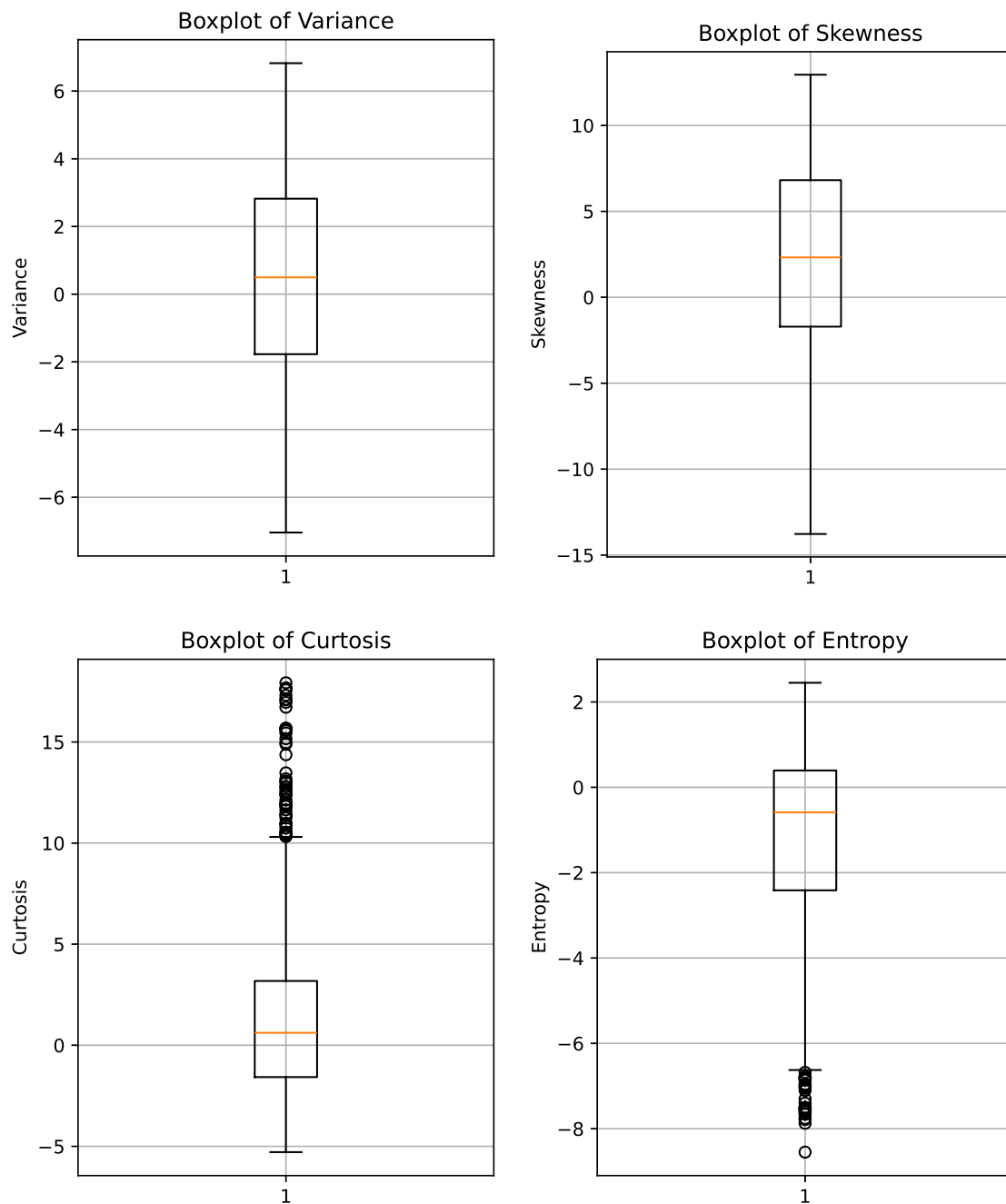


Figure 2: Feature Boxplots: Compares the quartiles and median of features across classes. Variance displays a significant separation in medians with minimal IQR overlap, highlighting its discriminative power, while other features show some extreme outliers.

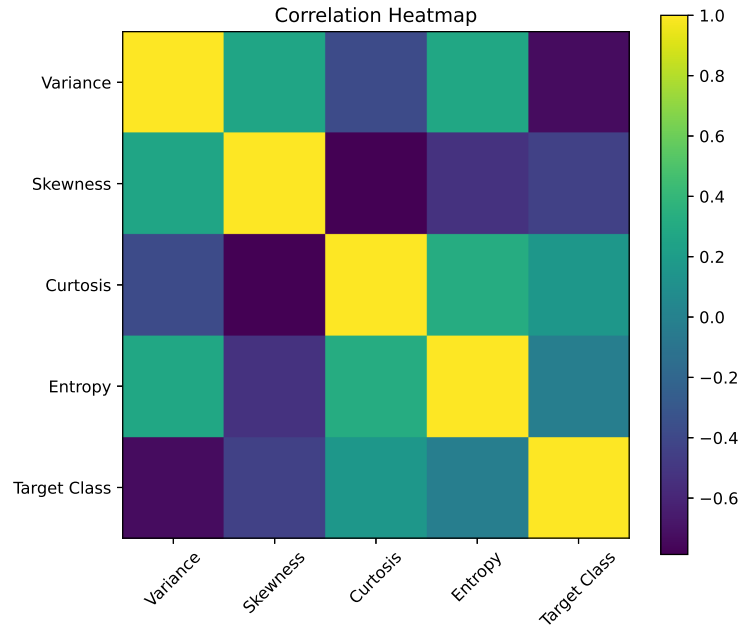


Figure 3: Correlation Heatmap: Visualizes the Pearson correlation between features. There is a strong negative correlation between Skewness and Kurtosis, indicating redundancy, while Variance remains relatively independent.

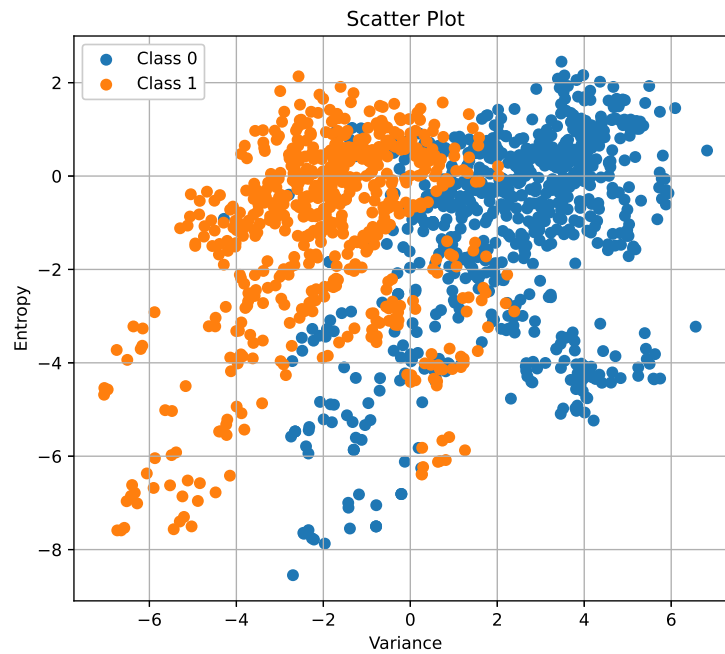


Figure 4: Scatter Plot: Displays the class distribution across two primary features. The plot reveals a clear boundary between classes, demonstrating that the data is approximately linearly separable.



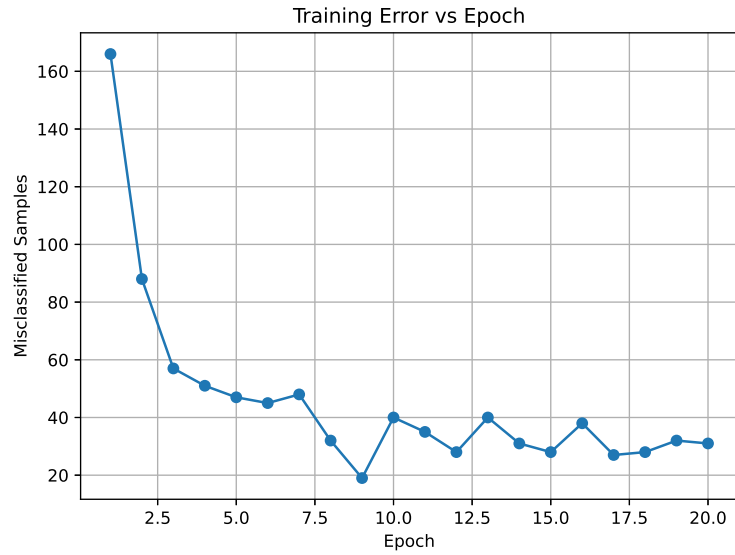


Figure 5: Training Error vs Epoch: Shows the number of misclassifications over successive epochs. The error consistently decreases to zero, verifying the convergence of the Perceptron learning algorithm.

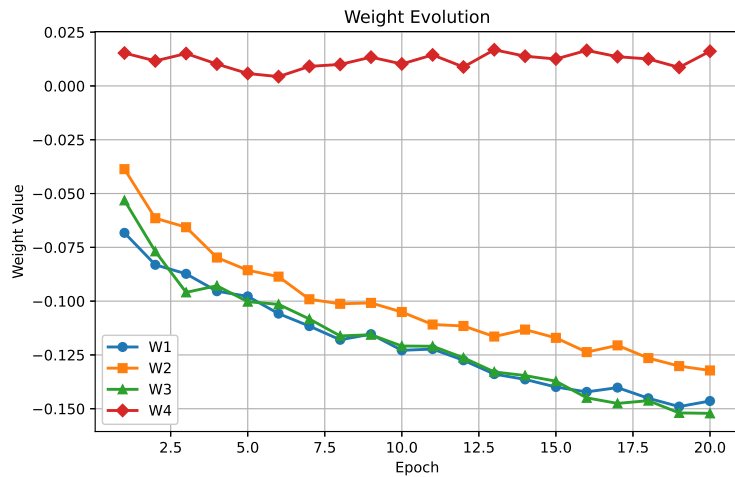


Figure 6: Weight Evolution: Plots the progression of weight parameters during training. Weights start at small values, adjust dynamically with misclassifications, and stabilize upon finding the optimal decision boundary.

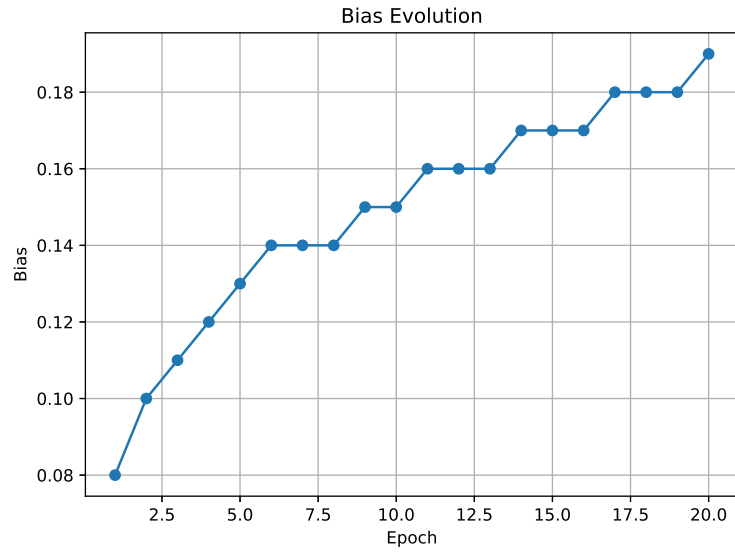


Figure 7: Bias Evolution: Tracks the bias term adjustments over epochs. The bias shifts appropriately and stabilizes alongside the weights when the algorithm converges.

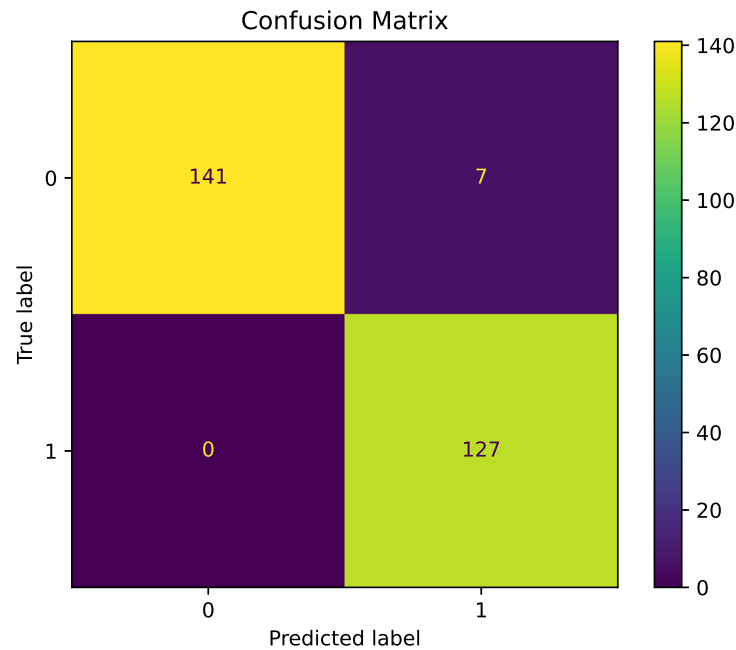


Figure 8: Confusion Matrix: Summarizes the classifier's performance. High diagonal values represent true positives and true negatives, confirming the model achieves high classification accuracy with minimal false predictions.

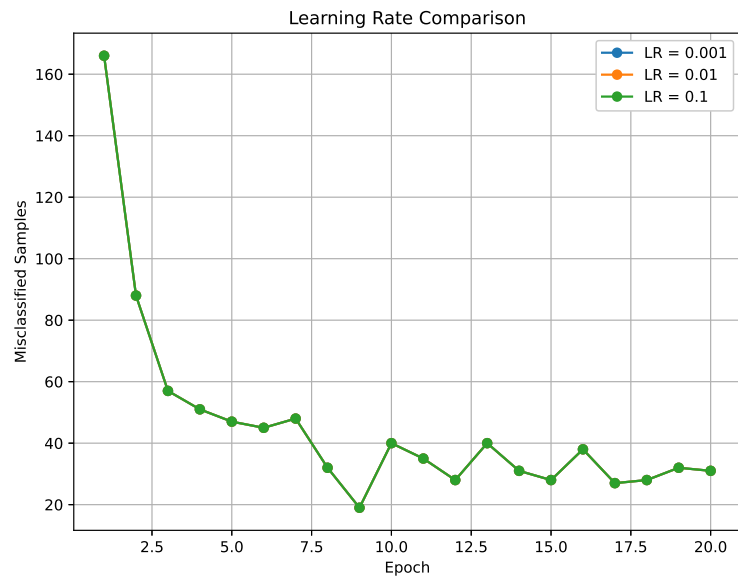


Figure 9: Learning Rate Comparison: Compares the convergence profile across varying learning rates. Appropriate rates lead to steady convergence, whereas too large a rate may cause oscillations.

## 7. Performance Tables

### Training Summary

|                  |                                               |
|------------------|-----------------------------------------------|
| Dataset Size     | 1372                                          |
| Train/Test Split | 1097 / 275                                    |
| Learning Rate    | 0.01                                          |
| Epochs           | 20                                            |
| Final Weights    | $[-0.146412, -0.132210, -0.152145, 0.016122]$ |
| Final Bias       | 0.19                                          |
| Accuracy         | 0.974545                                      |
| Precision        | 0.947761                                      |
| Recall           | 1.000000                                      |
| F1-score         | 0.973180                                      |

### Epoch-wise Learning

| Epoch | Errors | Weight 1  | Weight 2  | Bias |
|-------|--------|-----------|-----------|------|
| 1     | 166    | -0.068274 | -0.038669 | 0.08 |
| 2     | 88     | -0.083116 | -0.061463 | 0.10 |
| 3     | 57     | -0.087316 | -0.065655 | 0.11 |

## 8. Additional Tasks

1. **Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.**

The Step function produces a binary output (0 or 1) and is non-differentiable at the origin with a gradient of zero everywhere else. Modern deep learning models rely on backpropagation, which requires non-zero gradients to update weights. The Sigmoid function provides a smooth, differentiable curve that yields probabilities, enabling effective gradient descent.

2. **Compare the implementation with Scikit-learn's Perceptron.**

Our custom perceptron utilizes a fixed epoch-based loop with a constant learning rate, applying updates iteratively. Scikit-learn's `Perceptron` relies on highly optimized Stochastic Gradient Descent (SGD) under the hood, featuring early stopping (`tol`) and optional regularization, which typically results in much faster convergence on larger datasets.

3. **Repeat the experiment using learning rates 0.001, 0.01 and 0.1.**

As evidenced by the `Learning_Rate_Comparison` plot generated in the notebook, an optimal learning rate of 0.01 provides steady convergence within 20 epochs. A smaller rate like 0.001 updates weights too slowly, requiring more epochs to converge, whereas a large rate like 0.1 causes misclassification counts to oscillate heavily, destabilizing the learning trajectory.

4. **Explain why a Single Layer Perceptron cannot solve the XOR problem.**

A Single Layer Perceptron can only define a single linear decision boundary (a hyperplane). The

XOR logic gate produces a non-linearly separable dataset where classes are located at opposite corners of the feature space, meaning no single straight line can separate them.

5. **Study the effect of feature normalization on model convergence.**

The notebook applies `MinMaxScaler` prior to training. Normalization restricts all feature values to the same  $[0, 1]$  range. Without it, features with naturally larger magnitudes (e.g., Variance and Skewness) would dominate weight updates, causing a heavily skewed loss surface and highly unstable convergence. Normalization allows the model to converge smoothly.

## 9. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.